

Super-Resolution for Video Games

DSBA 6165



Eric Phann | Samantha Michael | Kidus Kidane

Table of Contents

1

Introduction/Problem Statement

2

Literature Review

3

Datasets/Code

4

Methodology/Plan

5

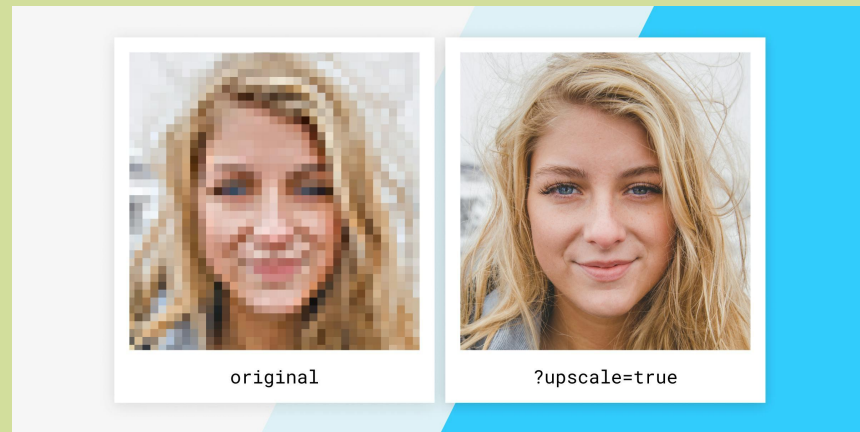
Learning Outcomes

Introduction

Image Super-Resolution is a machine learning task where the goal is to increase the resolution of an image, often by a factor of 4x or more, while maintaining its content and details as much as possible.

The end result is a high-resolution version of the original image. This task can be used for various applications such as improving image quality, enhancing visual detail, and increasing the accuracy of computer vision algorithms.

source: <https://paperswithcode.com/task/image-super-resolution>



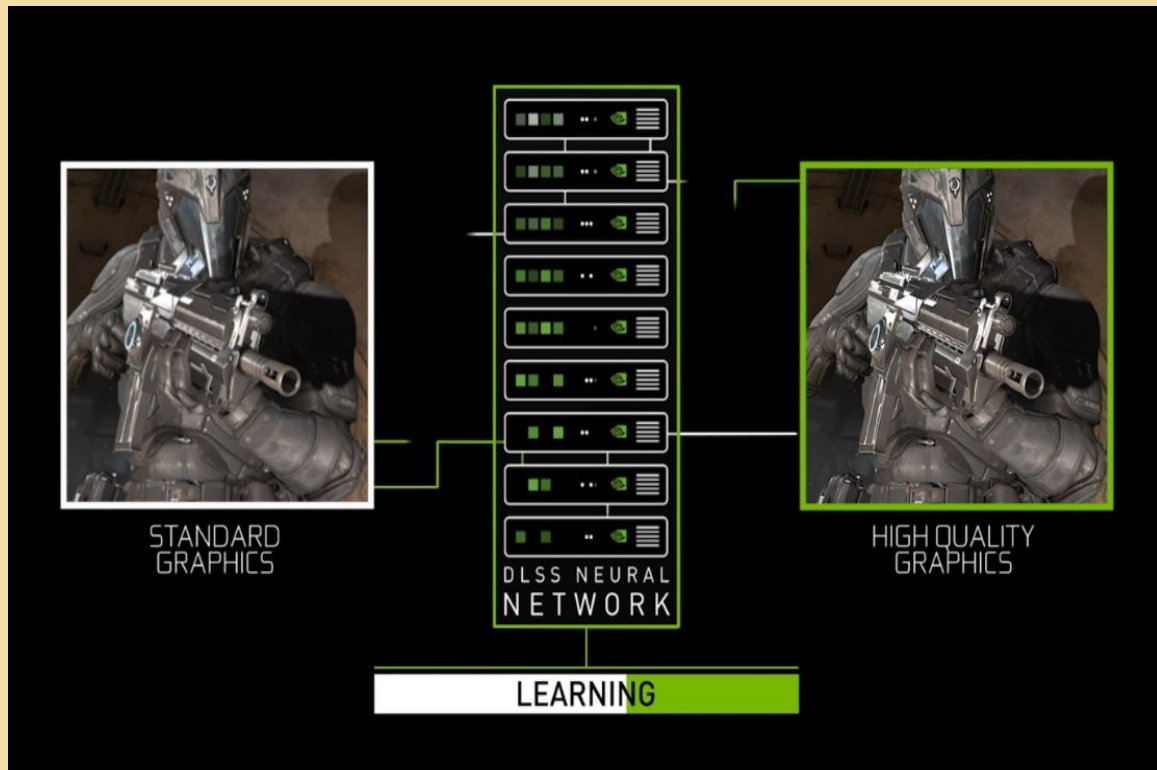
src: <https://www.imgix.com/jp/blog/ai-powered-image-super-resolution>

Deep Learning Super Sampling (DLSS)

Problem Statement

Video games are getting increasingly more intensive with each generation.

NVIDIA's GPUs dominate the market and boast DLSS, which aims to improve GPU efficiency and graphical fidelity via neural networks. As gamers, we'd like to see more open source and democratized models geared towards Image Super-Resolution in the video game industry.



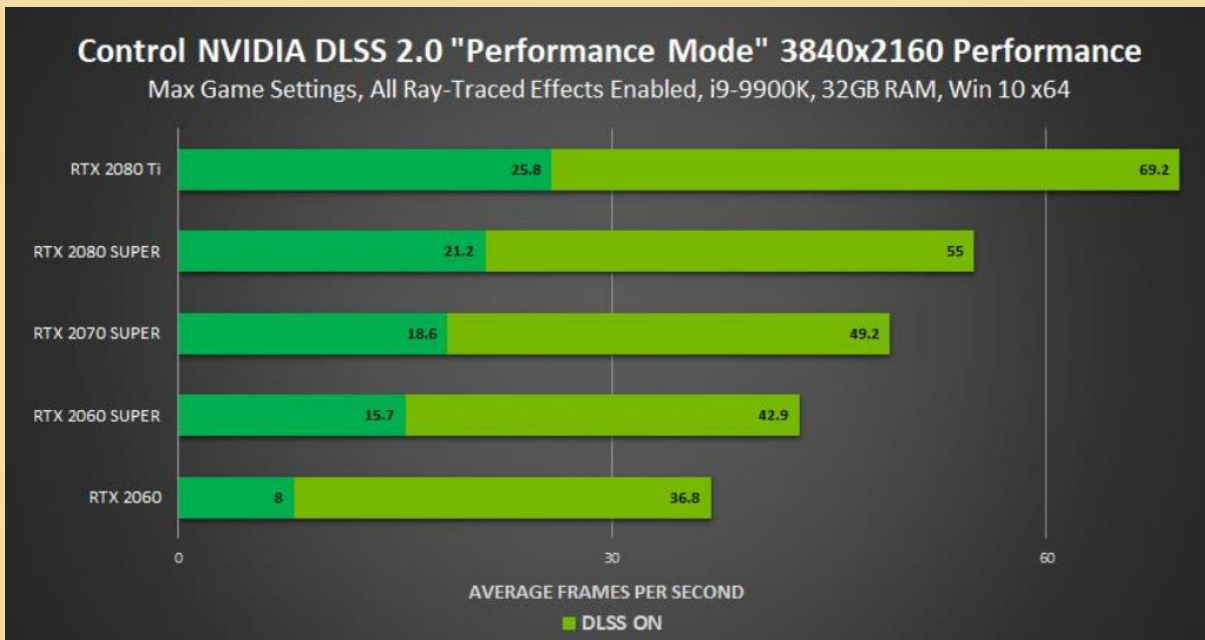
Deep Learning Super Sampling (DLSS)

Motivation

Common use case:

- 1) Run the game at a lower resolution (better performance), then...
- 2) Upscale to native resolution via DLSS ("better" graphics)

This is much more computationally efficient than running at native res. It allows lower-end GPU users to access and enjoy new, graphically intense titles.



Existing Approaches

Deep Learning for Super Resolution: Architectures Growing Deeper Over Time

- SRCNN (2014) → Pioneer → 3 Convolution Layers
- VDSR (2016) → Going Deeper → 20 Convolutional Layers
- SRGAN (2017) → Complex Architecture
→ Generator with 16 residual blocks + Discriminator
- ERSKAN (2018) → Modern Approach → Very deep generators (30+ layers) with dense connections

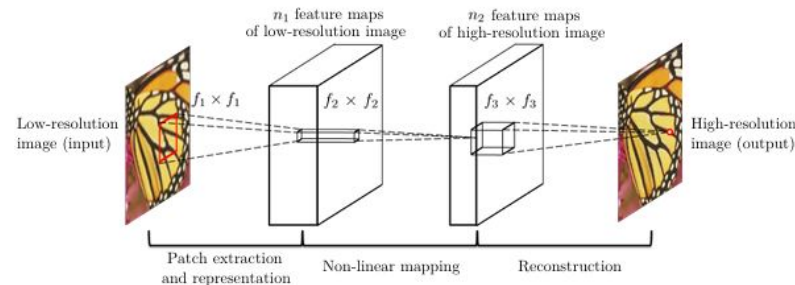
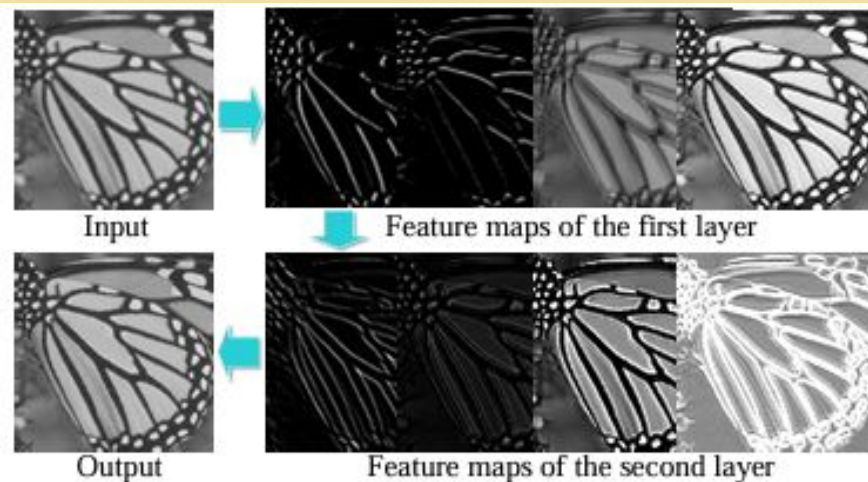


Convolutional Networks

- First major application of CNNs to super resolution (2014)
- Shallow network with simple architecture (3 layers)
- Demonstrated superior results to traditional methods
- Established basic framework for deep learning-based SR
- Limited by its shallow architecture and pre-upsampling, causing inefficiency and poor feature capture

Section 2

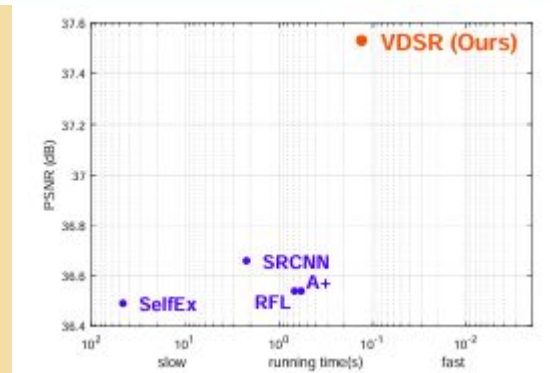
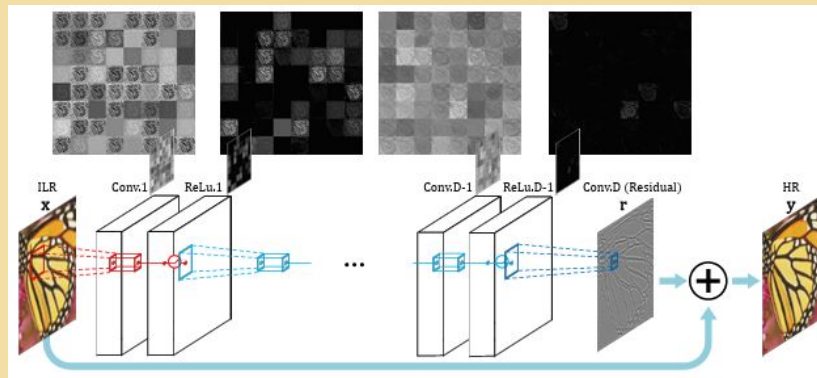
SRCNN - Image Super-Resolution Using Deep Convolutional Networks



Very Deep Super Resolution (VDSR)

- 20-layer VGG network for SR (2016)
- Demonstrated how deeper networks improve SR performance (20 layers)
- Deep residual learning prevents vanishing gradients
- Achieved state-of-the-art results in speed and accuracy outperforming SRCNN
- Faster convergence through gradient clipping
- Prioritizes pixel accuracy over perceptual quality, producing unrealistic textures

Accurate Image Super-Resolution Using Very Deep CNNs



Generative Adversarial Networks

SRGAN (2017)

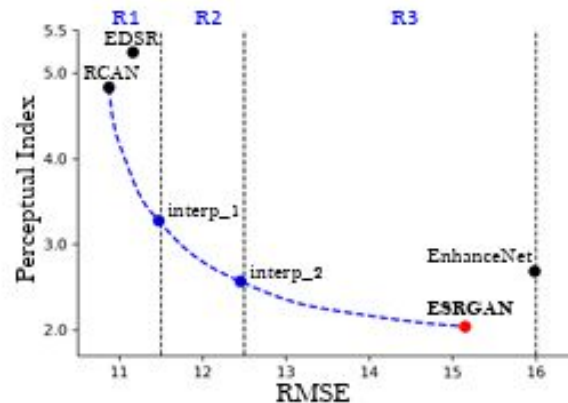
- Introduced adversarial training for more realistic results
- Perceptual loss instead of just pixel-wise loss
- Generator with 16 residual blocks
- Produced more photorealistic results than previous methods
- Can sometimes generate artifacts or hallucinate details that weren't in the original image



Enhanced GAN

ESRGAN (2018)

- Combines enhanced residual learning with GAN architecture
- Very deep generator network (30+ layers) with dense connections
- Improved training stability compared to SRGAN
- Better perceptual quality in generated high-resolution images
- Computationally intensive and slower than simpler models, making real-time applications challenging



Open Source Code

Available Open Source Codes:

<https://github.com/yin870/SRCNN-pytorch?tab=readme-ov-file>

SRCNN – Image Super-Resolution Using Deep Convolutional Networks

```
1  from torch import nn
2
3
4  class SRCNN(nn.Module):
5      def __init__(self, num_channels=1):
6          super(SRCNN, self).__init__()
7          self.conv1 = nn.Conv2d(num_channels, 64, kernel_size=9, padding=9 // 2)
8          self.conv2 = nn.Conv2d(64, 32, kernel_size=5, padding=5 // 2)
9          self.conv3 = nn.Conv2d(32, num_channels, kernel_size=5, padding=5 // 2)
10         self.relu = nn.ReLU(inplace=True)
11
12     def forward(self, x):
13         x = self.relu(self.conv1(x))
14         x = self.relu(self.conv2(x))
15         x = self.conv3(x)
16         return x
```

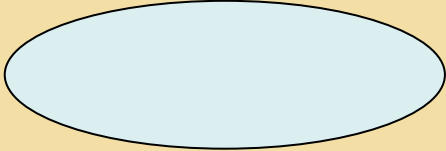
How to contribute:

1) Improve performance:

Experiment with deeper architectures or advanced functions

2) Optimized for use Cases

Add multi-channel (RGB) support



Methodology

We plan to each take a separate approach to the problem of Image Super Resolution, based on the previous literature reviews, for a total of 3 final models:

- Convolutional Neural Network (CNNs)
- Generative Adversarial Network (GANs)
- Transformers

We will each analyze and explore a specific dataset and come to a consensus about which datasets are ideal for each model or if we use one single dataset across all.

We will then create baseline models and improve upon them over the coming weeks, focusing on implementation and innovation where possible, over accuracy.

Dataset

- **Super Resolution in Video Games** – A training dataset consisting of paired 270p and 1080p images collected from projects using the Unreal Game engine.

<https://github.com/epishchik/SRGD>

LR



HR



VDSR Implementation ~ Baseline

Very Deep Convolutional Networks

Input:

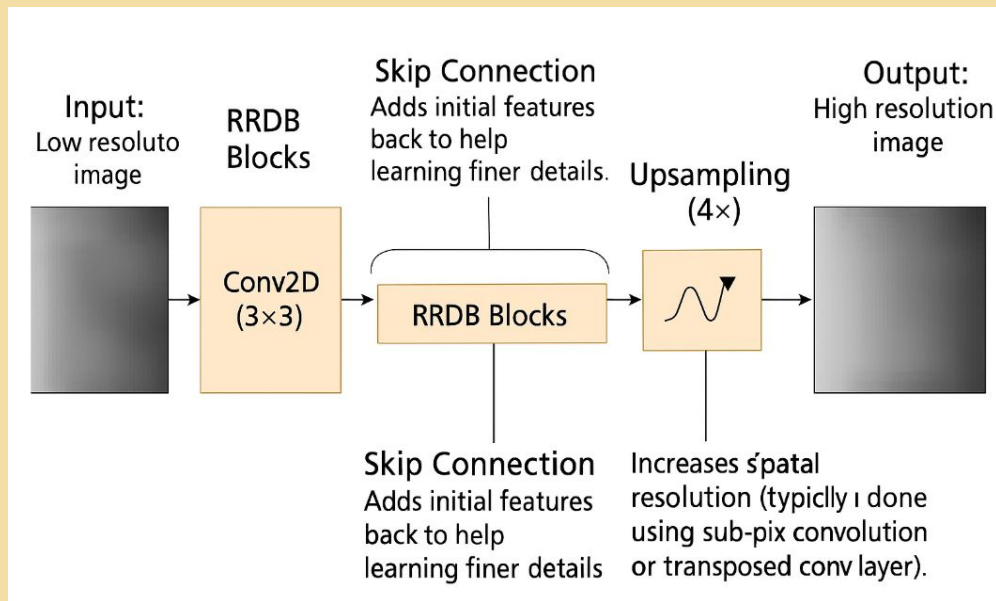
- Low-Resolution (LR) image (grayscale, normalized [0,1])

Layers:

- **Conv2D (3×3, 64 filters, ReLU):**
Initial feature extraction.
- **18 Conv2D Layers (3×3, 64 filters, ReLU each):**
Deep nonlinear transformations capturing complex image features.
- **Conv2D (3×3, 1 filter, no activation):**
Predicts the **residual** (difference between HR and interpolated L).

Output:

- High-Resolution (HR) reconstructed image.



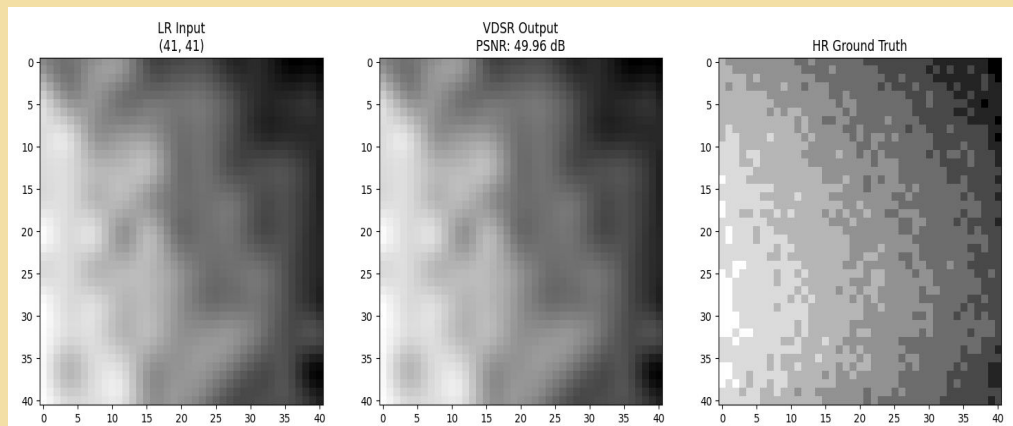
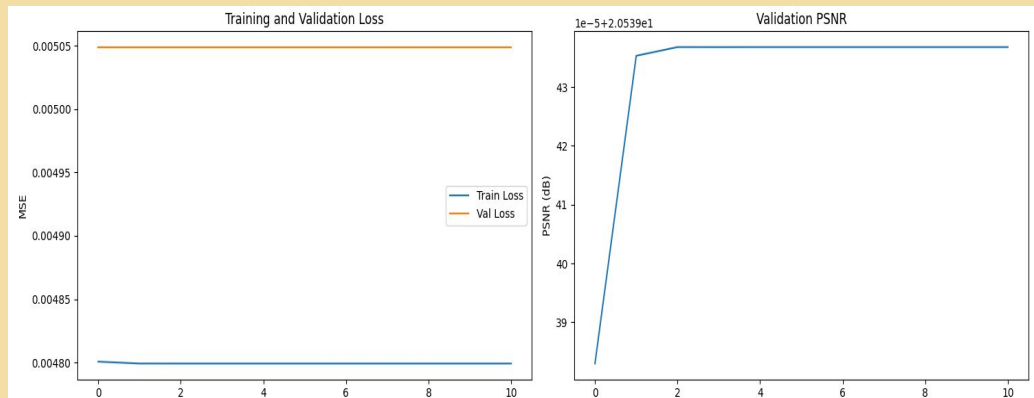
Results

Very Deep Convolutional Networks

Training

- Loss plateaued quickly (~ 0.0050 MSE), showing stable convergence.
- Early stopping triggered after 10 epochs without validation improvement.
- Final Test PSNR: **26.20 dB** on unseen test set.
- Output images were smooth, but some fine textures were missing.

 Final Evaluation:
Test PSNR: 26.20 dB



VDSR Implementation ~ Fine Tuned

Fine Tuned Very Deep Convolutional Networks

Input:

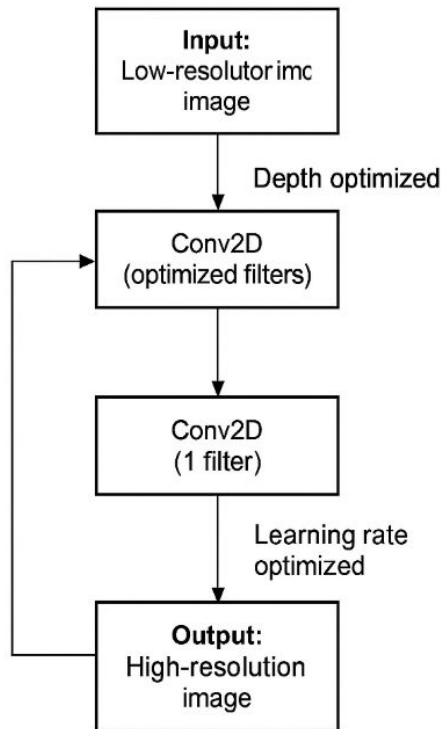
- Low-Resolution (LR) image (grayscale, normalized [0,1])

Layers:

- **Conv2D (3×3, 64 filters, ReLU):**
Initial feature extraction.
- **33 Conv2D Layers (3×3, 64 filters, ReLU each):**
Deep feature transformations (based on tuned depth=35).
- **Conv2D (3×3, 1 filter, no activation):**
Predicts residual (difference between HR and interpolated LR)

Output:

- High-Resolution (HR) reconstructed image with same dimensions as input.



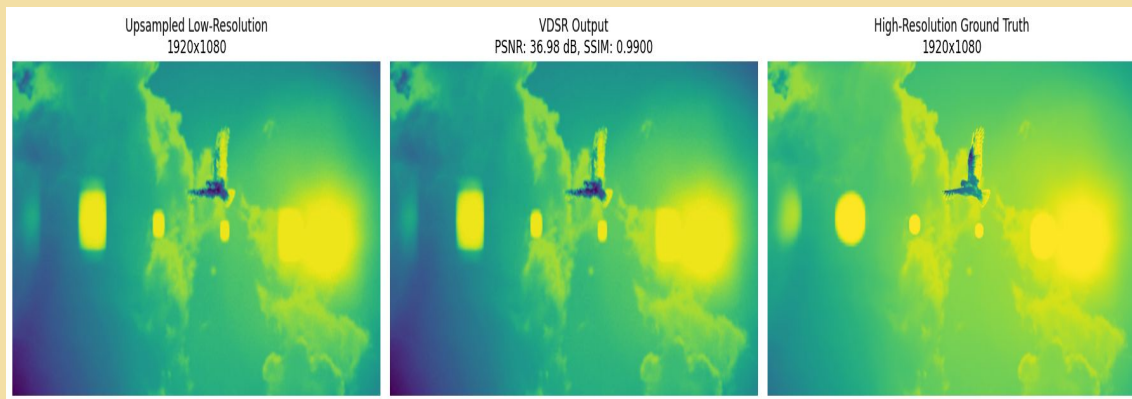
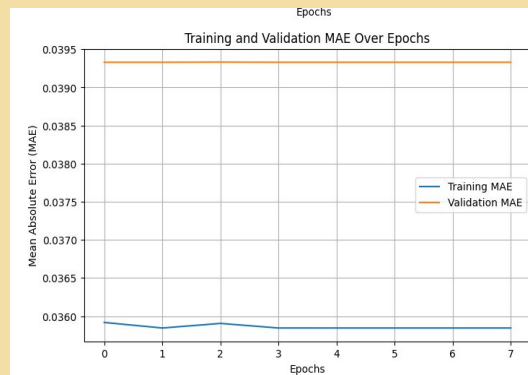
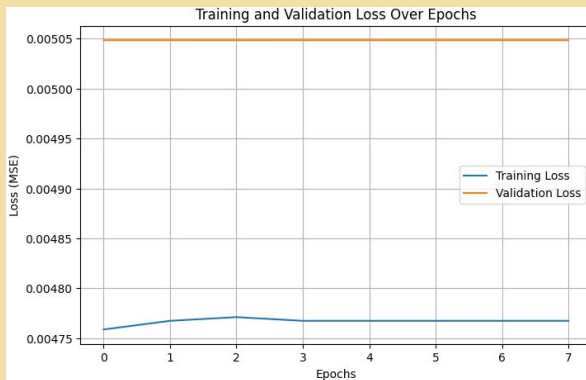
Fine Tuning Results

Fine Tuned Very Deep Convolutional Networks

Training

- Training and validation loss plateaued quickly (~0.0050 MSE).
- Early stopping triggered after ~8 epochs to prevent overfitting.
- Final Test PSNR: **30.74 dB** (up from **26.20 dB** baseline).
- Visual results improved, but **still resembled upsampled LR** in some areas despite high PSNR/SSIM.
- High pixel similarity (PSNR/SSIM) **does not always guarantee** better perceived visual quality

✓ Test PSNR: 30.74 dB
✓ Test SSIM: 0.7574



Generator Overview

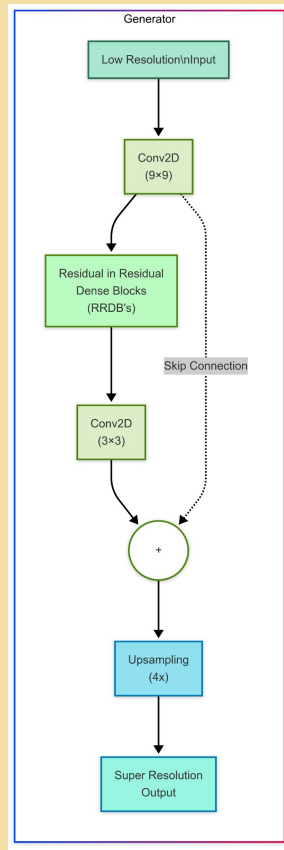
Generative Adversarial Network

Input: Low resolution image

Layers:

- **Conv2D (9×9):** Initial feature extraction.
- **RRDB Blocks (Residual in Residual Dense Blocks):** Deep feature transformation with enhanced skip connections for stable training.
- **Conv2D (3×3):** Post-RRDB transformation.
- **Skip Connection:** Adds initial features back to help learning finer details
- **Upsampling (4×):** Increases spatial resolution (typically done using sub-pixel convolution or transposed conv layers).

Output: High resolution image



Discriminator Overview

Generative Adversarial Network

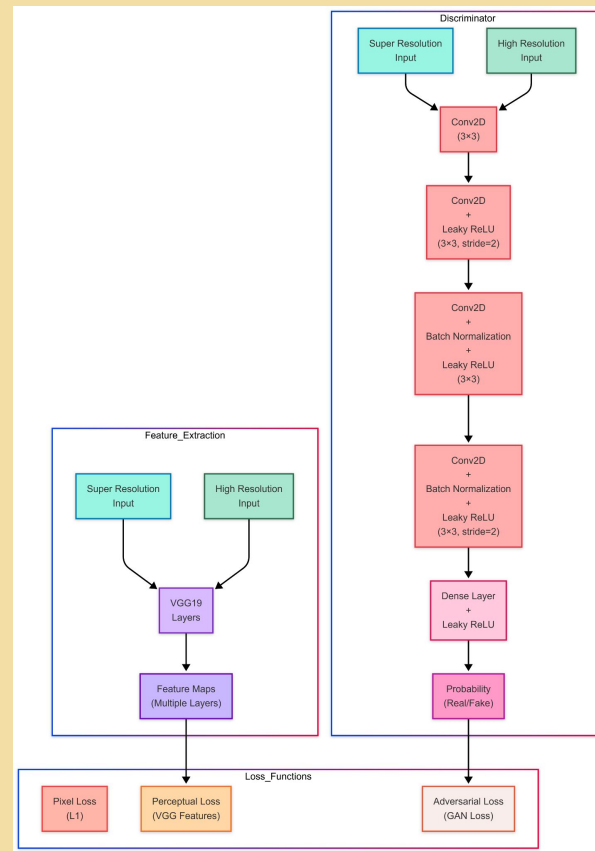
Input: Both real high-resolution & generated images.

Layers:

- **Stacked Conv2D layers:** With LeakyReLU, batch normalization, and stride=2 to reduce spatial dimensions.
- **Dense Layer:** Followed by LeakyReLU.
- **Output:** A probability indicating whether the image is real or generated.

VGG19:

- **Feature Extraction**
- **Input:** Both generated and real high-resolution images.
- **Output:** Extracted feature maps from intermediate layers to compute **perceptual loss**.



GAN Loss Functions

Generator Loss Components

- **Perceptual Loss:** VGG-based feature differences between HR and generated images
- **Pixel Loss:** Mean absolute error (L1) between HR and generated images
- **Adversarial Loss:** Binary cross-entropy measuring generator's ability to fool discriminator

Total:

$\text{gen_total_loss} = (\text{perceptual_loss_weight} \times \text{perceptual_loss}) + (\text{pixel_loss_weight} \times \text{pixel_loss}) + (\text{adversarial_loss_weight} \times \text{adversarial_loss})$

Discriminator Loss

- **Real Loss:** BCE for correctly classifying real images
- **Fake Loss:** BCE for correctly classifying generated images

Total:

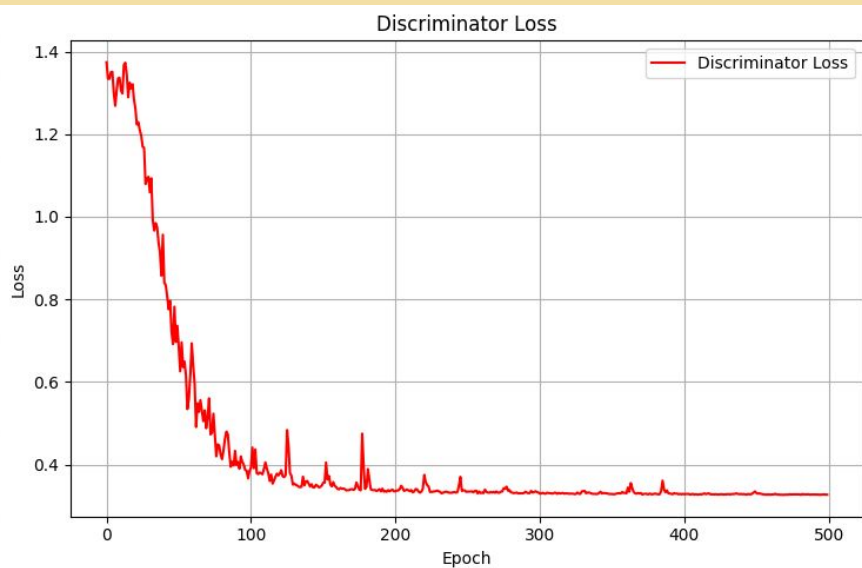
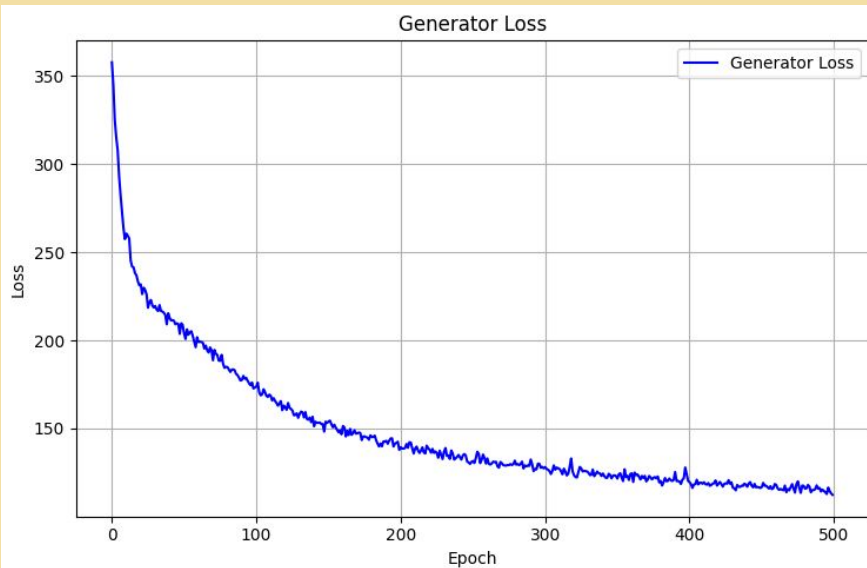
$\text{disc_loss} = \text{disc_loss_real} + \text{disc_loss_fake}$

Results - Baseline

Generative Adversarial Network

Training

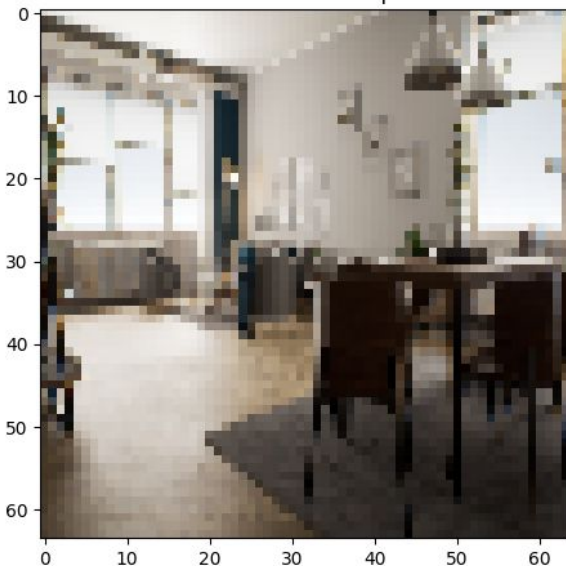
- 500 epochs in colab
- No weight initialization



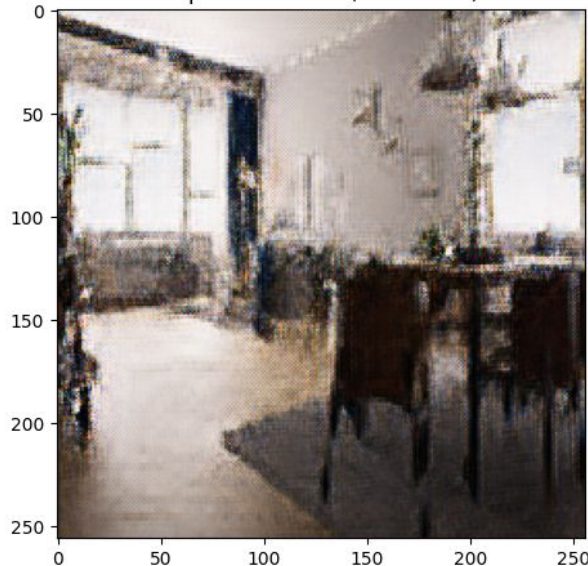
Results - Baseline

Generative Adversarial Network

Low Resolution Input



Super Resolution (Generated)



High Resolution (Ground Truth)



Hyperparameter Tuning

Generative Adversarial Network

- We ran 10 Optuna trials to tune the model
- Trial #7 produced the best results with a PSNR score of 19.70
- The optimal architecture used 8 residual blocks with 96 filters
- A moderate dropout rate of 0.297 helped prevent overfitting
- These settings achieved the best balance between perceptual quality and reconstruction accuracy

Best Trial #7 Results

PSNR: 19.69608497619629

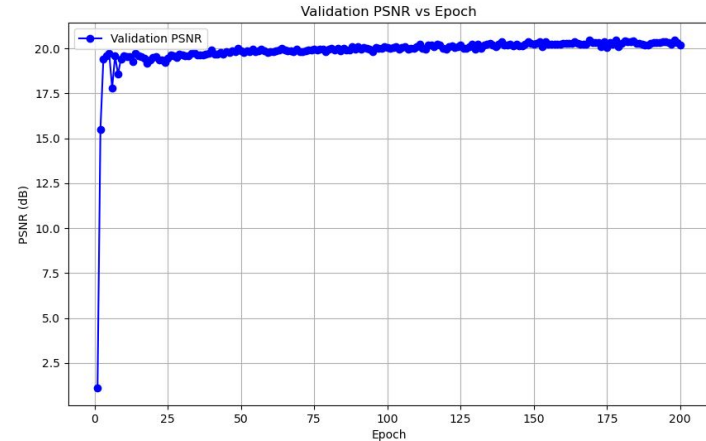
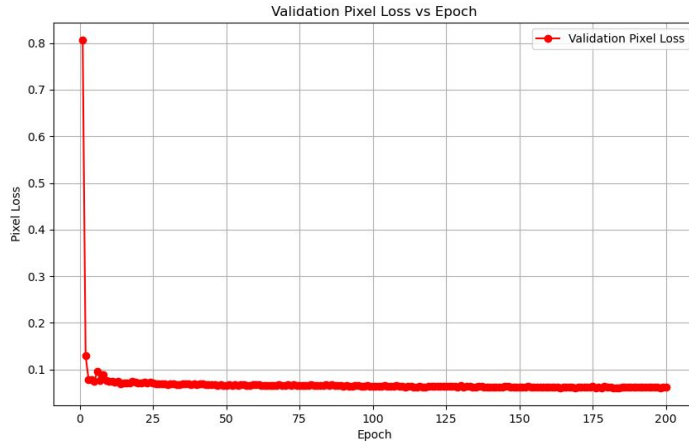
Hyperparameters

Parameter	Value
num_residual_blocks	8
filters	96
dropout_rate	0.2971465273570587
lr_gen	4.6750250331130036e-05
lr_disc	1.1942012783540352e-05
perceptual_loss_weight	0.028146790912123167
pixel_loss_weight	0.1481637014313447
adv_loss_weight	0.07625602618459658

Results - Final

Generative Adversarial Network

Performance Metric	Result (200 Epochs)
Average PSNR	19.86 dB
Average Pixel Loss	0.0695

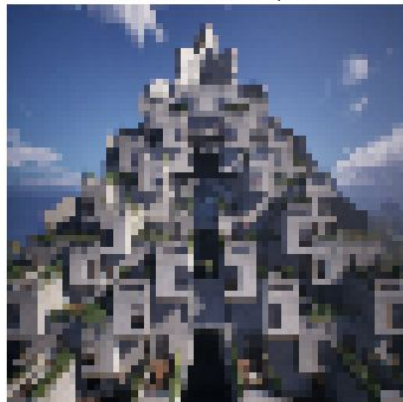


Generative Adversarial Network

- SRGAN Output is significantly lighter than the input and ground truth
- Implemented post-processing color correction after upscaling

Example 4

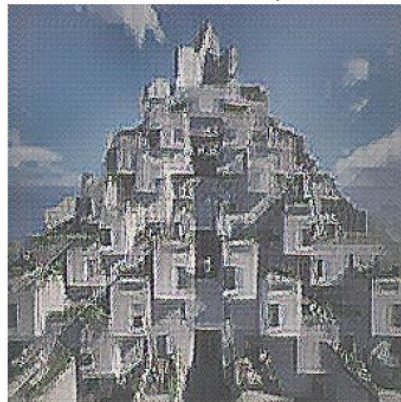
Low Resolution (Input)



Super Resolution (SRGAN Output)



Corrected SR Output



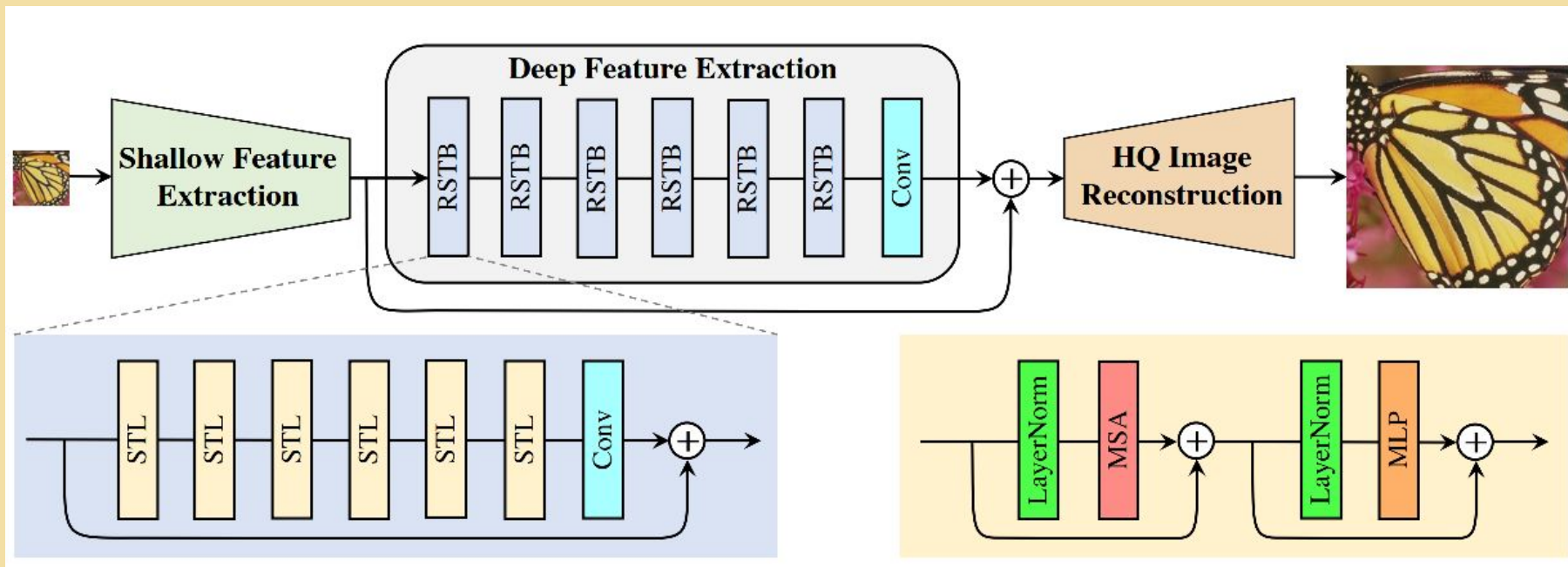
High Resolution (Ground Truth)



Model Overview

Transformers

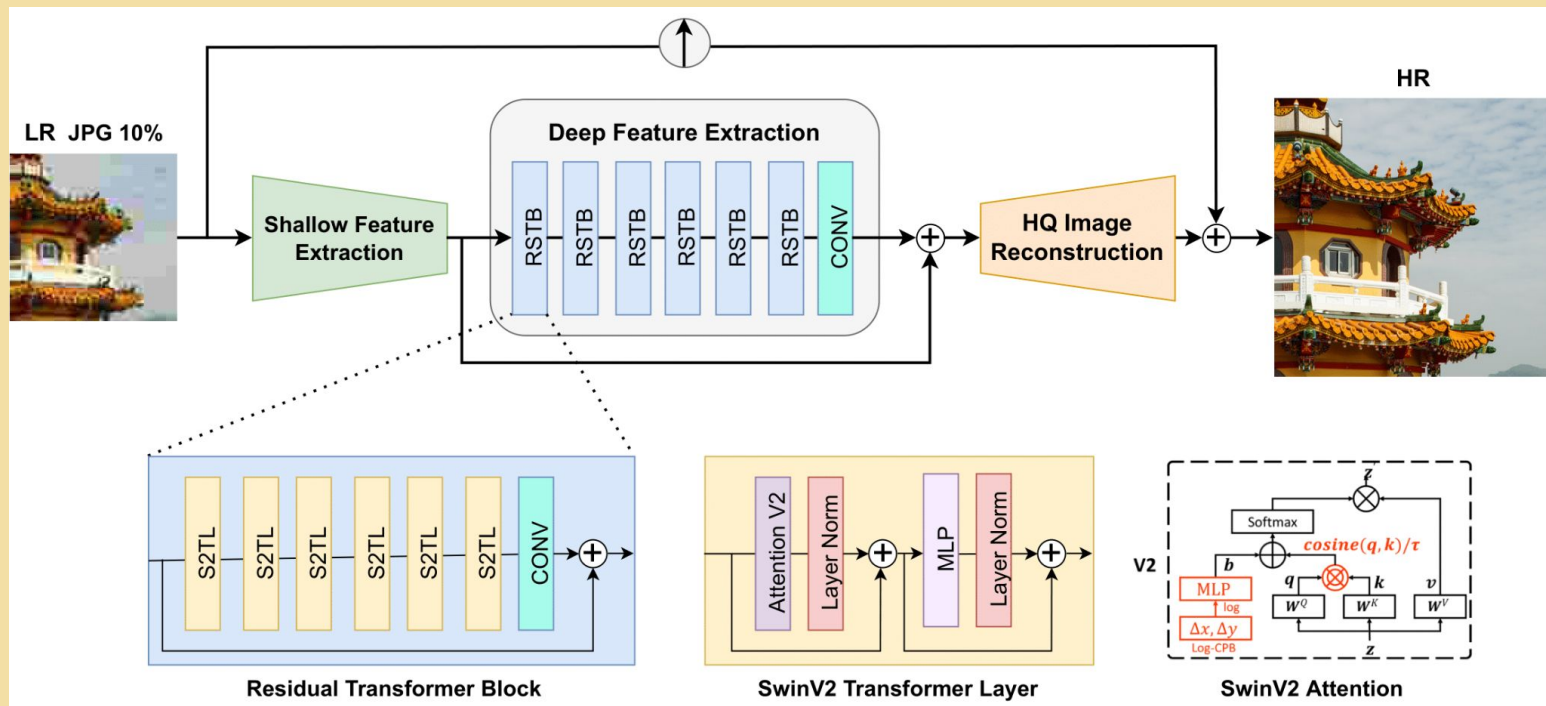
SwinIR: “Shifted **W**indow Image Reconstruction” composed of residual “swin” transformer blocks (RSTB) and “swin” transformer layers.



Model Overview

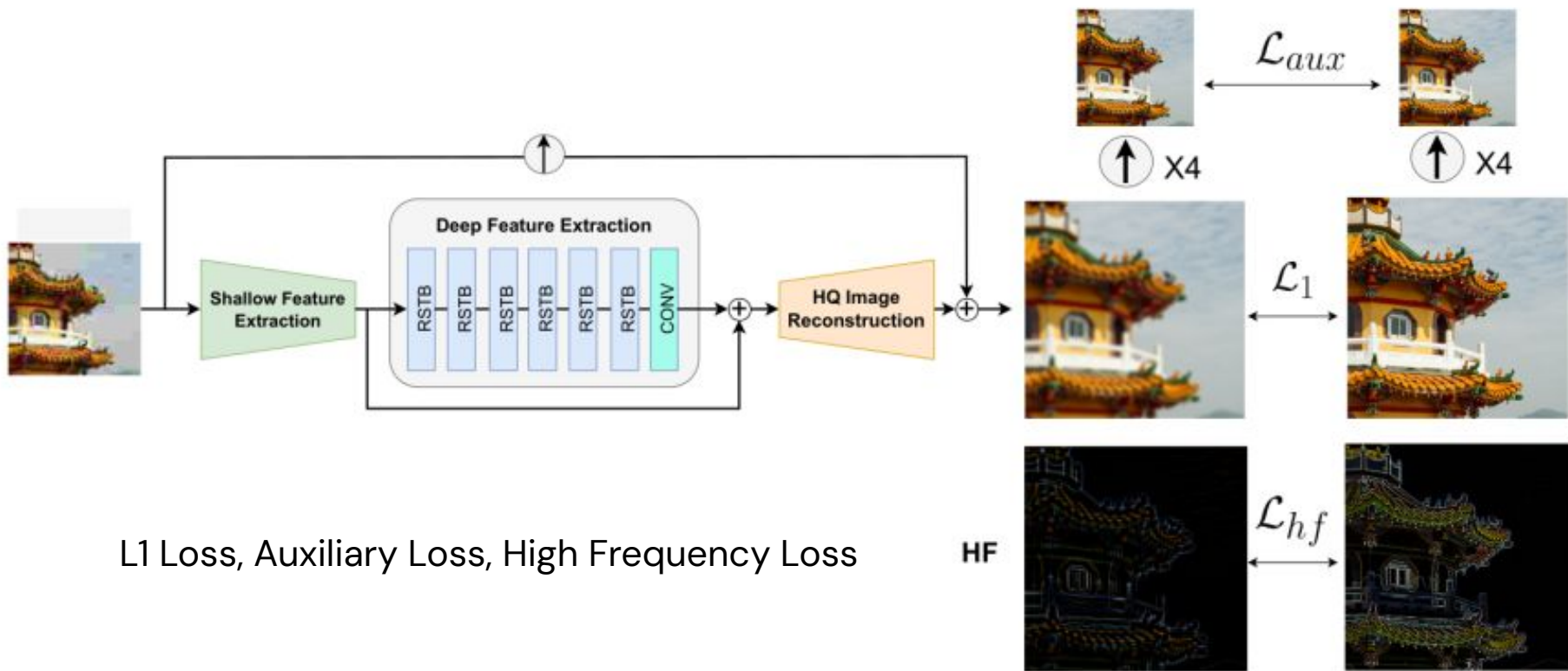
Transformers

SwinV2 Transformer: Updated to use post-normalization and scaled cosine attention, increasing performance on super resolution tasks by 33%



Cost / Loss

Transformers



L1 Loss, Auxiliary Loss, High Frequency Loss

Low-Res Input (Upscaled)



Ground Truth High-Res



Predicted High-Res



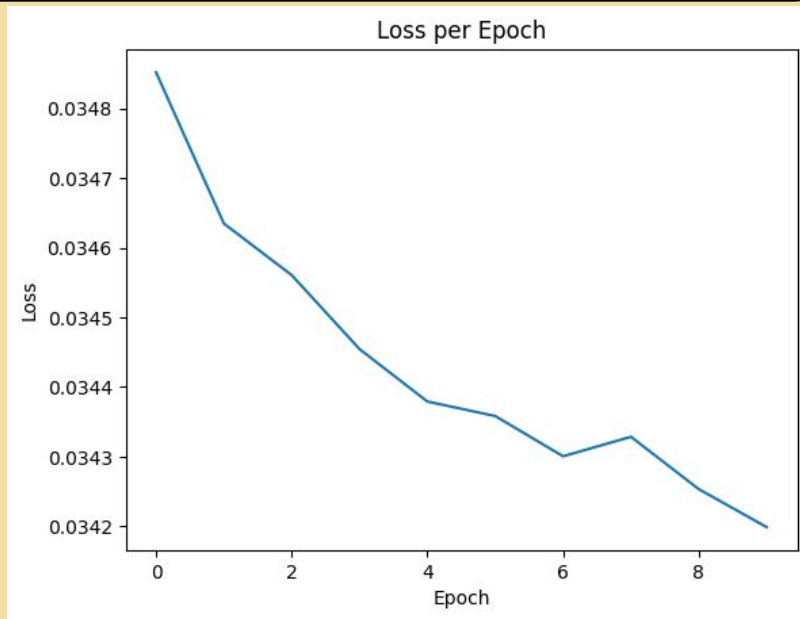
Peak Signal to Noise Ratio
(PSNR): **17.85 dB**

Structural Similarity
Index Measure (SSIM): **0.4825**

Fine-tuning

Transformers

- AdamW optimizer (direct weight decay)
- Charbonnier Loss (smoother L1 variant)
- 250 low-res/high-res image pairs from the Super Resolution Game Dataset
- Froze all layers for training except last 2 layers and subsampling layer
- Perceived visual quality may still increase despite PSNR/SSIM decrease (can be misleading for super resolution metric)

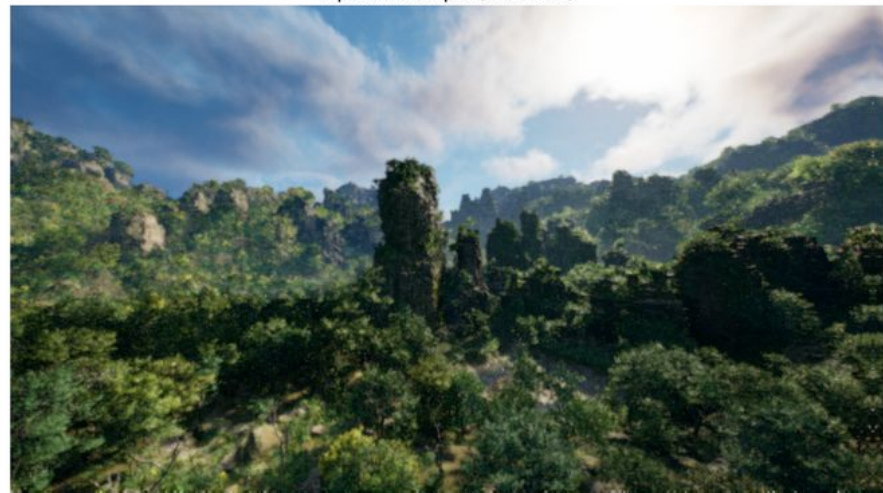


```
PSNR (Base-line): 25.05 dB
PSNR (Fine-tuned): 24.11 dB
SSIM (Base-line): 0.7671
SSIM (Fine-tuned): 0.7658
```

Low-Res Input



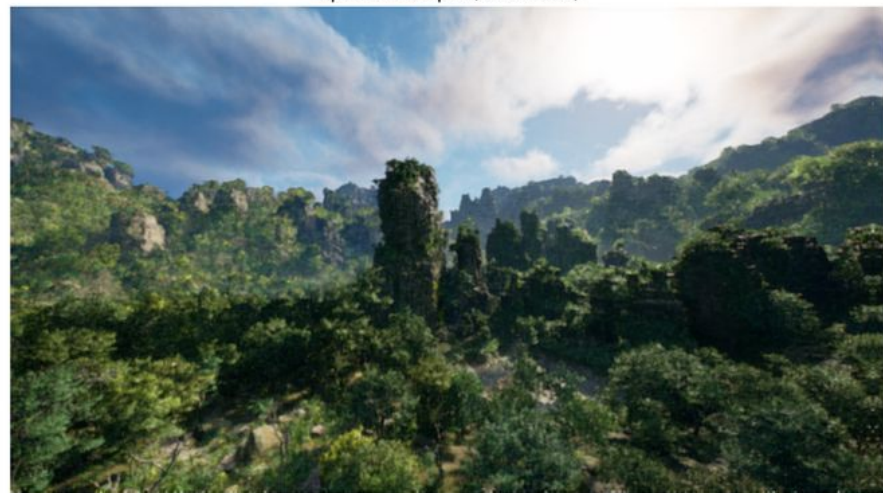
Upscaled Output (Base-line)



Ground Truth High-Res



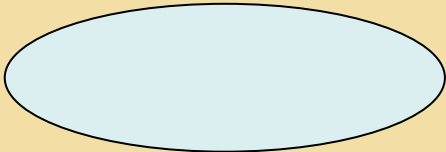
Upscaled Output (Fine-tuned)



Challenges

- Moving from the Colab environment to HPC Cluster
- Distributed Computing Issues
- Working with Complex Architectures
- Working with large image files and big datasets
- Pixel-wise losses (MSE) optimized numerical similarity but did not guarantee realistic fine details.
- Long training times
- State of the art super resolution transformers leveraged multiple loss functions to account for various image quality characteristics





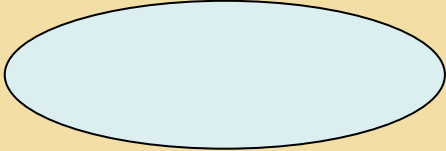
Conclusion

Learnings and insights

- Fine-tuning VDSR with Optuna hyperparameters significantly improved PSNR and training stability.
- Weight initialization makes a huge difference in training time
- Scaling an implementation to the hpc cluster is difficult

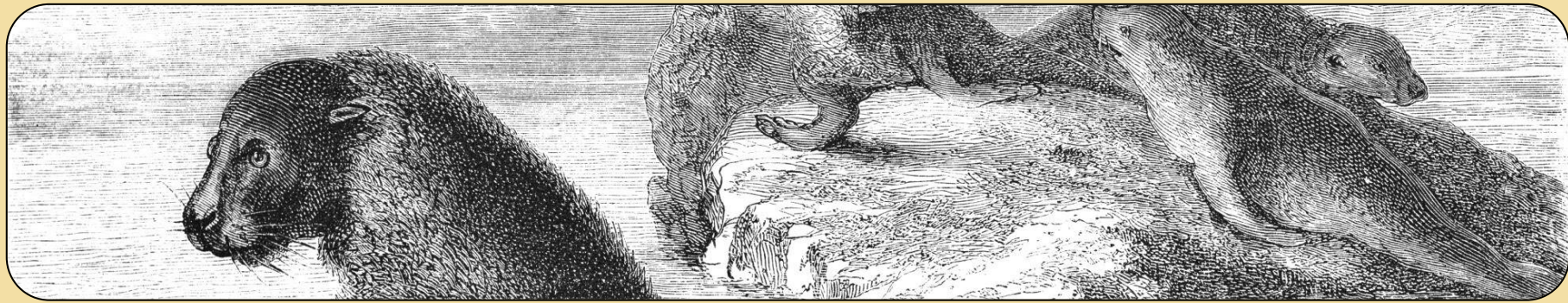
Future Work

- Implement early stopping to prevent overfitting
- Add more residual blocks (16-23 total) to the GAN to upgrade to ESRGAN architecture
- Experiment with different loss functions and unfreezing certain layers



References

- Conde, M. V., Choi, U.-J., Burchi, M., & Timofte, R. (2022, September 22). *Swin2SR: SwinV2 transformer for compressed image super-resolution and restoration*. arXiv.org. <https://arxiv.org/abs/2209.11345>
- Dong, C., Loy, C. C., He, K., & Tang, X. (2014). *Image super-resolution using deep convolutional networks*. arXiv. <https://arxiv.org/abs/1501.00092>
- Kim, J., Lee, J. K., & Lee, K. M. (2016). *Accurate image super-resolution using very deep convolutional networks*. arXiv. <https://arxiv.org/abs/1511.04587>
- Simonyan, K., & Zisserman, A. (2014). *Very deep convolutional networks for large-scale image recognition*. arXiv. <https://arxiv.org/abs/1409.1556>
- Wang, X., Yu, K., Wu, S., Gu, J., Liu, Y., Dong, C., Qiao, Y., & Change Loy, C. (2018). *ESRGAN: Enhanced super-resolution generative adversarial networks*. arXiv. <https://arxiv.org/abs/1809.00219>
- Wang, X., Xie, L., Dong, C., & Shan, Y. (2021). *Real-ESRGAN: Training real-world blind super-resolution with pure synthetic data*. arXiv. <https://arxiv.org/abs/2107.10833>



THANK YOU

